

Other Storage Options

Mr. Tan Chiang Kang



Other Storage Options

- SQLite is the oldest and well-known relational database for mobile applications development
- But, that is not the only choice
- Another alternative is an object-based solution, i.e. Room Persistent Library (Android) and Core Data (iOS)
- There is also another cross-platform solution known as Realm

Room Persistent Library

- SQLite is sometimes very tedious for the developer to use, requiring many similar codes, prone to errors and difficult to manage multiple queries
- The Room Persistent Library introduced by Google, add an abstraction layer for the existing SQLite API
- Objects and simpler notations are used to manipulate the database instead

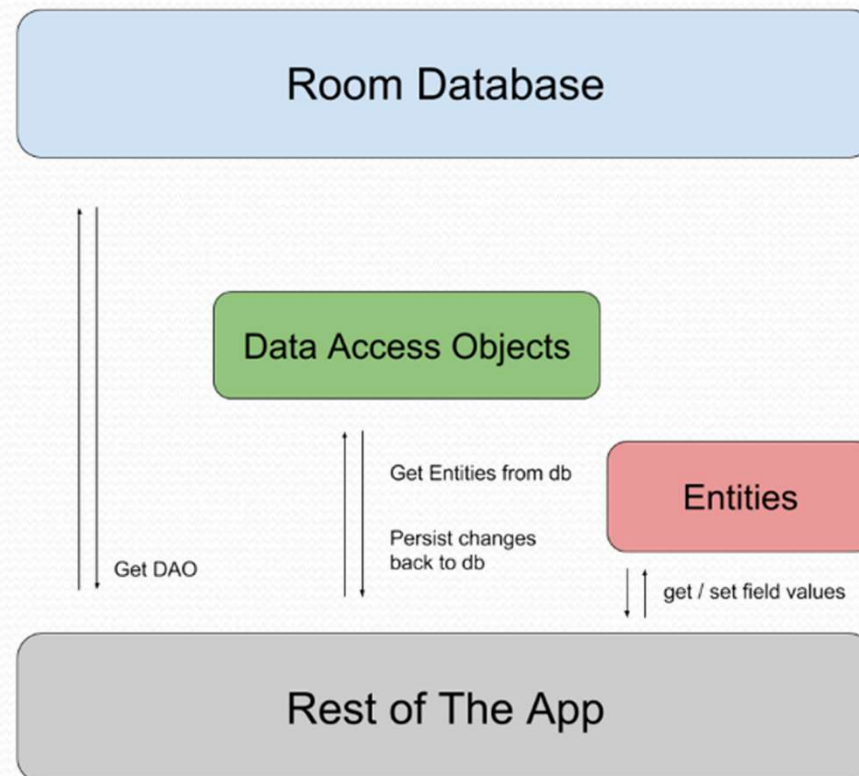
Room Persistent Library

- Annotations are used for the Room database

Annotations	Purpose
@Entity	Creates a SQLite table in the database using a data model class.
@Dao	Create a Data Access Object in the database using an interface class.
@Database	A class with this annotation will create an abstraction for the Data Access Object.
@PrimaryKey	A variable with this annotation will set a primary key for the table.
@Insert	Inserts parameters into the table.
@Update	Updates parameters of an existing table.
@Delete	Deletes parameters of an existing table
@Query	Running SQL query method within the table
@Ignore	Ignores the parameter form the Room database

Room Persistent Library

- Entity
- Data Access Objects (DAO)
- Room Database



Entity

- A class that represents a table in the database
- It allows fields of the tables to be declared as the attributes of the class
- Annotations are used to mark the primary key (e.g. `@PrimaryKey`), or override the name of the fields

Entity - Example

@Entity

```
public class Movies {  
    @NonNull  
    @PrimaryKey  
    private String movieId;  
    private String movieName;  
  
    public Movies() {  
    }  
  
    public String getMovieId() { return movieId; }  
    public void setMovieId(String movieId) { this.movieId =  
        movieId; }  
    public String getMovieName() { return movieName; }  
    public void setMovieName (String movieName) {  
        this.movieName = movieName; }  
}
```

Data Access Objects (DAO)

- An interface to work with the database, to perform CRUD operations (i.e. Create, Read, Update and Delete) on an entity
- Abstract methods are created for each operation
- Every query can be directly expressed as the SQL queries

DAO - Example

@Dao

```
public interface DaoAccess {
```

```
    @Insert
```

```
    void insertOnlySingleMovie (Movies movies);
```

```
    @Insert
```

```
    void insertMultipleMovies (List<Movies> moviesList);
```

```
    @Query ("SELECT * FROM Movies WHERE movieId = :movieId")
```

```
    Movies fetchOneMoviesbyMovieId (int movieId);
```

```
    @Update
```

```
    void updateMovie (Movies movies);
```

```
    @Delete
```

```
    void deleteMovie (Movies movies);
```

```
}
```

Room Database

- An abstract class that is a subclass of RoomDatabase
- The entities (tables) and version must be defined for the database
- The abstract DAO class will also be declared here
- When the database object is declared and initialized, it can make use of the abstract methods declared in the DAO to interact with the database

Room Database - Example

```
@Database (entities = {Movies.class}, version = 1,  
          exportSchema = false)  
public abstract class MovieDatabase extends RoomDatabase {  
  
    public abstract DaoAccess daoAccess() ;  
}
```

Declare and Initialization

- First declare and initialise an object for the Database class in the Activity or Fragment class

```
private static final String DATABASE_NAME =  
    "movies_db";  
private MovieDatabase movieDatabase;  
movieDatabase =  
    Room.databaseBuilder(getApplicationContext(),  
        MovieDatabase.class, DATABASE_NAME)  
        .fallbackToDesctructiveMigration()  
        .build();
```


Accessing the Database

- When the initial steps are done, you can now access the database via the database object

```
new Thread(new Runnable() {
    @Override
    public void run() {
        Movies movie =new Movies();
        movie.setMovieId("2");
        movie.setMovieName("The Prestige");
        movieDatabase.daoAccess().
            insertOnlySingleMovie(movie);
    }
}).start();
```

Advantages of Room over SQLite

- SQLite does not validate SQLite queries during compile time, Room does
- A lot of boilerplate codes are required in SQLite to convert between SQL queries and Java data objects, Room maps database objects to Java Object more easily
- Room can work with LiveData and RxJava but SQLite does not

Realm

- Realm is an open source object database management system, initially designed for mobile implementations, now also supports Xamarin, React Native, etc
- Realm supports Objective-C, Swift, Java and Kotlin, hence can be used in Android and iOS
- Advantage:
 - Easy to use
 - Very fast query speed
 - Convenient Realm Browser to view and manage
 - Cross-platform

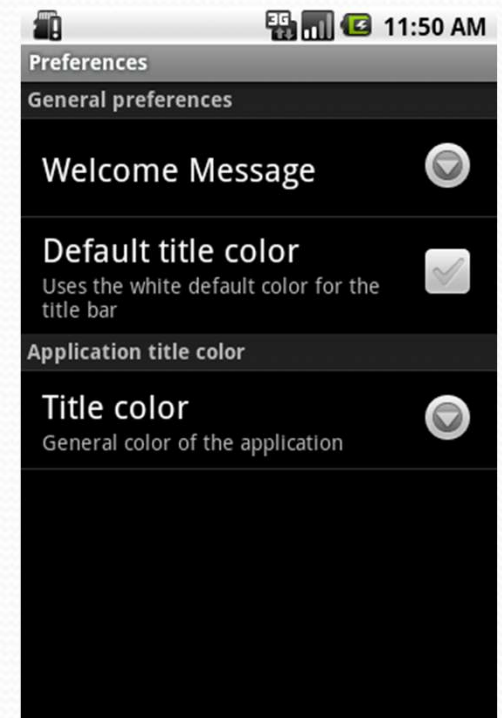
Other Storage Options

- Other than storing in databases (SQLite), modern mobile app platforms offer other options on storing and sharing data
 - Simple Data Pairs
 - File I/O



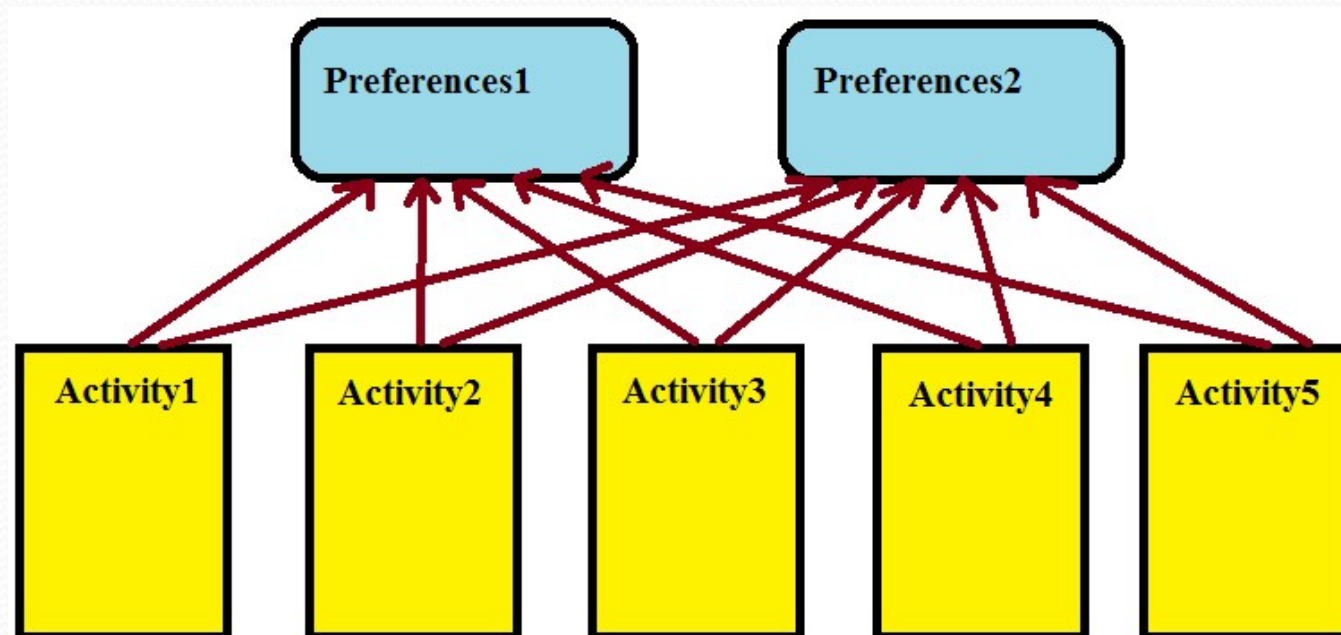
Simple Data Pairs

- Sometimes you do not need to store large amount of data, or that the data does not need to be stored in complex tabled form
- Simple data pairs is a very lightweight solution to store data for an application
- It is usually used to store “**preferences**” – e.g. an application state, configuration option, simple user information, a url bookmark and etc
- Examples: Android (**Shared Preferences**) and iOS (**NSUserDefaults**)



Shared Preferences

- Shared Preferences is only shared among **Activities** in the same package
- Preferences are stored as groups of key/value pairs



Supported Data Types

- Some common data types are supported to be stored as preferences:
 - Boolean values
 - Float values
 - Integer values
 - Long values
 - String values

SharedPreferences Interface

- To use share preferences, the **SharedPreferences** interface is used
- Required to import **android.content.SharedPreferences**
- An editor is needed to alter the data of the shared preferences using **SharedPreferences.Editor**

Steps to Use Preferences

- To add preferences to the application, the following steps are required:
 - Retrieve an instance of a **SharedPreferences** object
 - Create a **SharedPreferences.Editor** to modify preference content
 - Make changes to the preferences using the Editor (add, update and delete)
 - Commit your changes

Creating Preferences

- Preferences can be private or shared
- The private preferences are restricted for the private use of the Activity:

```
SharedPreferences prefPrivate =  
    getPreferences (MODE_PRIVATE) ;
```

- The shared preferences can be used in any activity of the application through the preference name:

```
SharedPreferences pref =  
    getSharedPreferences (  
        "MySharedPreferences", 0) ;
```


Reading Preferences

- The preferences can be read from the **SharedPreferences** objects
- Each preference can be accessed by its name and provided with a default value if not found
- The preferences are accessed by different data type
- Some methods are available for easy access of data from the preferences

SharedPreferences Methods

- Common methods for SharedPreferences
- **contains()** – sees if a specific preference exists by name/key
- **edit()** – retrieves the editor to change these preferences
- **getAll()** – retrieves a map of all key/value pairs representing the preferences

SharedPreferences Methods

- **getBoolean()** – retrieves a specific Boolean-type preference by name
- **getFloat()** – retrieves a specific Float-type preference by name
- **getInt()** – retrieves a specific Integer-type preference by name
- **getLong()** – retrieves a specific Long-type preference by name
- **getString()** – retrieves a specific String-type preference by name

Altering Preferences

- One can add, update or delete preferences
- To make changes, a preference **Editor** is required from **SharedPreferences.Editor**
- The Editor is created for a created **SharedPreferences** object

```
SharedPreferences.Editor prefEditor =  
    pref.edit();
```


Editor Methods

- **clear()** – removes all preferences. This operation happens first, regardless of when it is called within an editing session. Then all other changes are made and committed.
- **remove()** – removes a specific preference by name. This operation happens first, regardless of when it is called within an editing session. Then all other changes are made and committed.
- **commit()** – commits all changes from this editing session

Editor Methods

- **putBoolean()** – sets a specific Boolean-type preference by name
- **putFloat()** – sets a specific Float-type preference by name
- **putInt()** – sets a specific Integer-type preference by name
- **putLong()** – sets a specific Long-type preference by name
- **putString()** – sets a specific String-type preference by name

Using Editor

- After creating the **Editor** from existing **SharedPreferences**
- Multiple action (e.g. add or update preference) can be executed
- The changes will only take place when the editing is committed using **commit()**

- Example:

```
prefEditor.putLong("SomeLongPref", 0);  
prefEditor.commit();
```

Where is Preferences Stored?

- The preferences must be stored as a form of file so that it could be accessed by your application when you launch it again
- The application preferences are stored as XML files
- The file can be accessed by a Dalvik Debug Monitor Service (DDMS) using the File Explorer
- The preferences will normally be stored according to directory structure as follows:

```
/data/data/<package name>/shared_prefs/  
<preferences filename>.xml:
```


Preferences XML



- The format for the XML is as follows:

```
<?xml version="1.0" encoding="utf-8" standalone="yes"
  ?>
<map>
  <string name="String_Pref">Test String</string>
  <int name="Int_Pref" value="1234567890" />
  <float name="Float_Pref" value="12.34" />
  <long name="Long_Pref" value="1234567890987654321" />
  <boolean name="Boolean_Pref" value="false" />
</map>
```

Accessing Preferences XML

- Knowing the file format of the XML is helpful for testing purposes
- DDMS can be used to copy the preference files to and from the device
- It can be used for debugging purposes



Storing in Files

- Android application data are stored on the Android file system in the following directory:
`/data/data/<package name>/`
- As you have noticed by now, some default subdirectories are used for storing database and preferences as well as files
- You can also create custom directories when the need arises



File & Directory Management

- Some **Context** (**android.context.Context**) methods for file and directory management are as follows:
- **openFileInput()** – opens a file associated with this Context's application package for reading
- **openFileOutput()** – opens a file associated with this Context's application package for writing
- **deleteFile()** – deletes an application file by name

File & Directory Management

- **fileList()** – gets a list of all files associated with this Context's application package
- **getFilesDir()** – gets the path of the directory holding application files
- **getCacheDir()** – gets the path of the directory holding application cache files
- **getDir()** – retrieves, creating if needed, a new directory in which the application can place its own custom data files

Creating and Writing Files

- The **openFileOutput()** method will open (or create) a file to be written on
- The file written will be in the default **/files** subdirectory
- The opened file can be directly written on with **MODE_PRIVATE** chosen
- The opened file can be appended with data after the existing data with **MODE_APPEND**



Creating and Writing Files

- The **FileOutputStream** object is used to manage the writing of file
- Requires to import **java.io.FileOutputStream**
- When you open a file for writing, errors can occur
- The files may have errors in opening, cannot be found, and failed to be written on etc
- These processes are required to be placed in try/catch statements to ensure they do not crash the whole running application



Reading File

- To read a file, the file is opened as an input using **openFileInput()** instead
- Use **FileInputStream** object to manage the file for reading
- Import from **java.io.FileInputStream**
- To read in primitive Java data type, **DataInputStream** can be used (**java.io.InputStream**)
- The data can be read and appended in a **StringBuffer** that can be translated as **String** to be displayed or used

Reading XML files

- XML files have specific structure formed by using tags
- Android SDK includes several utilities for working with XML files, e.g. Simple API for XML (SAX)
- Helpful packages include:
 - `android.sax.*`
 - `android.util.Xml.*`
 - `org.xml.sax.*`
 - `javax.xml.*`
 - `org.w3c.dom`
 - `org.xmlpull.*`



Working with Directories

- The **openFileOutput()** and **openFileInput()** methods discussed previously are only for storing in the **/files** subdirectory
- You can control the directory yourself using **java.io.File** class
- The subdirectory can be created as a **File** object to be used to refer to the directory path
- This allows stronger file management by creating separate subdirectories for different scenarios

Writing in Cache

- Cache can be used to speed up performance of your app by storing some data temporarily on your device
- The cache are stored in cache files that can be created and stored
- In Android, the cache are stored under the following directory:

`/data/data/<package name>/cache/`

Writing in External Storage

- In order to read and write files on external storage (SDCard), permission should be granted
- READ_EXTERNAL_STORAGE for reading files
- WRITE_EXTERNAL_STORAGE for writing files
- Example:

```
<uses-permission android:name=  
"android.permission.WRITE_EXTERNAL_STORAGE" />
```


Check Media Availability

- In order to read/write on an external storage, the media must be there
- Android provides a method **getExternalStorageState()** to check if the media is available
- The method returns if the media is available for read/write
- It may also return other states: media is being shared (connected to a computer), is missing entirely, had been removed badly, etc

Save files to be shared

- Files can also be selected to be saved in public directories that matches the type of the data, such as **Music/**, **Pictures/**, and **Ringtones/**
- This allows other applications to easily browse the corresponding data contents via **getExternalStoragePublicDirectory()** method
- This will effectively differentiate similar type of data for different purposes, for example ringtones are not considered as music

What's Next?

- Data sharing
- Android networking